Step 1 — A Classifier That Sends Each Ticket to the Right Reply Chain — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — Two chains, not one
- Hint 2 — The plan, and almost the whole thing

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — Two chains, not one

This step is two small chains, one after the other:

- a **classifier**: `prompt | model.with_structured_output(TicketCategory)` — it returns a checked object, not text (Doc05's Intermediate exercise);
- a **reply chain**: `prompt | model | StrOutputParser()` — it returns plain text (Doc05's Basic exercise).

The classifier's `category` decides **which** reply prompt the second chain uses. That choice is the branch — a plain `if`, or a prompt file with the same name as the category.

`TicketCategory` in `schemas.py`:

```
category: Literal["billing", "technical", "general"]
reason:   str     # one sentence, for the log
sure:     bool    # False if it could fit more than one
```

`Literal` means the model can't invent a fourth category — Doc04's structured-output topic.

Name the reply prompts `billing.txt`, `technical.txt`, `general.txt`. Then `load_prompt(category)` picks the right one with no `if` at all.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
classify_ticket(ticket):
    chain = prompt(classifier.txt) | model.with_structured_output(...)
    try: result = chain.invoke({"ticket": ticket})
    except ValidationError: result = general, sure=False
    log category, sure, reason
    return result

handle_ticket(ticket, customer_id):
    category = classify_ticket(ticket)
    chain = build_reply_chain(category.category)
    reply = chain.invoke({"ticket": ..., "customer_id": ...})
    print the reply
    return TicketRecord(customer_id, ticket, category, reply)
```

Where each piece lives:

- `chains/classifier.py` — `build_classifier_chain()` and `classify_ticket()`;
- `chains/replies.py` — `build_reply_chain(category)`;
- `schemas.py` — `TicketCategory` and `TicketRecord` (4 fields for now);
- `services/pipeline.py` — `handle_ticket()`, returns a `TicketRecord`;
- `main.py` — reads `data/sample/tickets.json`, loops, `print_record()`; catches `AuthenticationError` (exit) and `RateLimitError` (skip the ticket).

Every reply chain gets the same input dict, even when its prompt uses only `{ticket}`: a template silently ignores extra keys (Doc05's Edge cases exercise).

Full solution: Show me the solution